LAPORAN PRAKTIKUM PENGENALAN APLIKASI BERBASIS INTERNET

PHP TINGKAT MENENGAH (INTERMEDIATE)

Agus Pranata Marpaung

13323033

DIII TEKNOLOGI KOMPUTER

INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI

Judul Praktikum

Minggu/Sesi	:	III/2
Kode Mata Kuliah	:	1331204
Nama Mata Kuliah	:	PENGENALAN APLIKASI BERBASIS INTERNET
Setoran	:	*Softcopy*
Batas Waktu Setoran	:	*9 Februari 2024 jam 17:00 WIB*
Tujuan	:	*1. Students Know how to implement page template using include() and require() method*

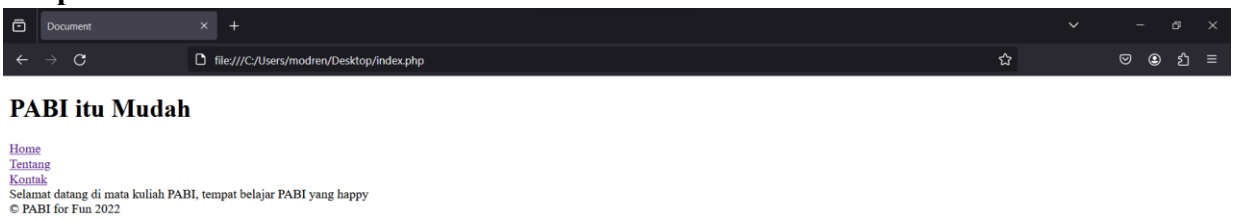
PHP Tingkat Menengah (Intermediate)

Membuat template halaman dengan Include dan Require

Ciptakan sebuah file **index.php** dengan konten sebagai berikut:

```
index.php > html
<!--
Nama    :
NIM     :
Kelas   :
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initialscale=1.0">
    <title>Document</title>
</head>
<body>
    <header>
        <h1> PABI itu Mudah </h1>
    </header>
    <nav>
        <li><a href ="index.php"> Home </a></li>
        <li><a href ="tentang.php"> Tentang </a></li>
        <li><a href ="kontak.php"> Kontak </a></li>
    </nav>
    <main>
        Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
    </main>
    <footer>
        &copy; PABI for Fun 2022
    </footer>
</body>
</html>
```

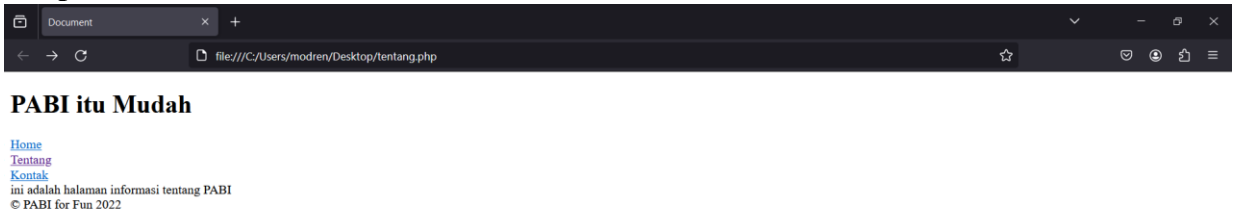
Output:



Selanjutnya, Ciptakan sebuah file **tentang.php** dengan konten sebagai berikut:

```
tentang.php > html
<!--
Nama    :
NIM     :
Kelas   :
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <header>
        <h1> PABI itu Mudah </h1>
    </header>
    <nav>
        <li><a href ="index.php"> Home </a></li>
        <li><a href ="tentang.php"> Tentang </a></li>
        <li><a href ="kontak.php"> Kontak </a></li>
    </nav>
    <main>
        ini adalah halaman informasi tentang PABI
    </main>
    <footer>
        &copy; PABI for Fun 2022
    </footer>
</body>
</html>
```

Output:

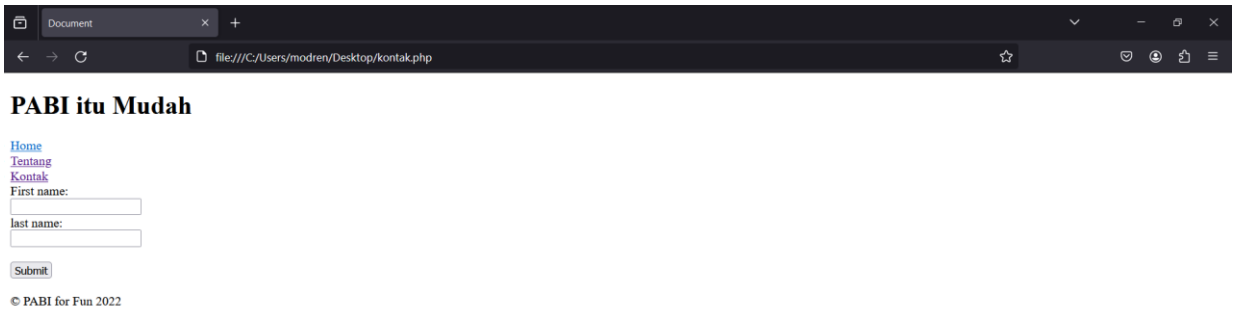


Selanjutnya, Ciptakan sebuah file **kontak.php** dengan konten sebagai berikut:

```
kontak.php > html
<!--
Nama    :
NIM     :
Kelas   :
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <header>
        <h1> PABI itu Mudah </h1>
    </header>
    <nav>
        <li><a href ="index.php"> Home </a></li>
        <li><a href ="tentang.php"> Tentang </a></li>
        <li><a href ="kontak.php"> Kontak </a></li>
    </nav>
```

```
    <main>
    <form action="">
        <label for="fname">First name:</label><br>
        <input type="text" id="fname" name="fname" value=""><br>
        <label for="lname">Last name:</label><br>
        <input type="text" id="lname" name="lname" value=""><br><br>
        <input type="submit" value="Submit"> <br><br>
    </form>
    </main>
    <footer>
        &copy; PABI for Fun 2020
    </footer>
</body>
</html>
```

Output:



Pada program diatas anda akan dapat mengakses konten dari halaman lain pada halaman yang sama yaitu **index.php.**

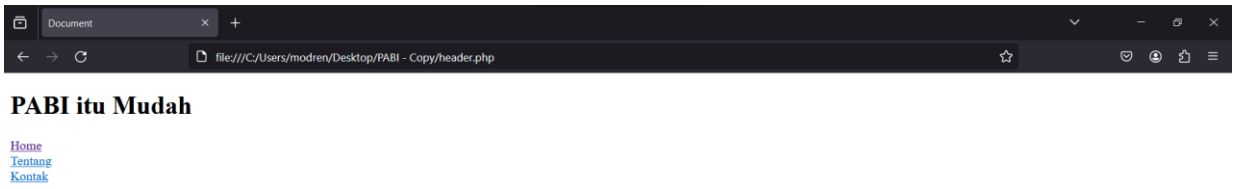
Bagaimana seandainya anda ingin mengganti sebuah baris program yang sama disetiap program anda. Contohnya saja baris program **"© PABI for Fun 2022"** ingin diubah menjadi **"© PABI for Fun 2023" .** apa yang harus user lakukan jika hendak merubah bagian ini pada program lain? Solusinya adalah anda harus menggantinya pada setiap halaman **(index.php, tentang.php dan kontak.php).** Tentunya ini bukan lah program yang efisien dan melanggar asas **DRY (Don't Repeat Your self)**

Maka solusi terbaiknya, PHP menyediakan fungsi **include() , require() dan require_once().** Mari kita praktikumkan sekarang.

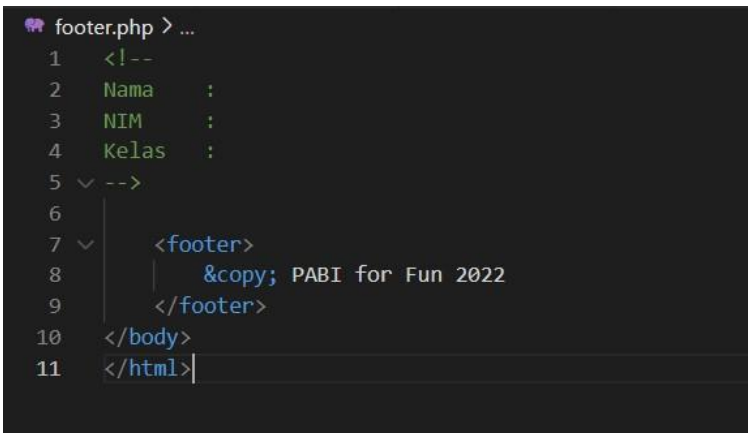
Pindahkan/cut potongan code dari **index.php** berikut kedalam satu file baru dengan nama: **header.php**

```
header.php > html > body > nav
1  <!--
2  Nama    :
3  NIM     :
4  Kelas   :
5  -->
6
7  <!DOCTYPE html>
8  <html lang="en">
9  <head>
10     <meta charset="UTF-8">
11     <meta name="viewport" content="width=device-width, initial-scale=1.0">
12     <title>Document</title>
13 </head>
14 <body>
15     <header>
16         <h1> PABI itu Mudah </h1>
17     </header>
18     <nav>
19         <li><a href ="index.php"> Home </a></li>
20         <li><a href ="tentang.php"> Tentang </a></li>
21         <li><a href ="kontak.php"> Kontak </a></li>
22     </nav>
```

Output:



Pindahkan/cut potongan code dari **index.php** berikut kedalam satu file baru dengan nama: **footer.php**



Output:

sehingga konten pada file **index.php** menjadi seperti berikut:

```
<!--
Nama    :
NIM     :
Kelas   :
-->

    <main>
        Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
    </main>
```

Jika anda ingin menampilkan konten **header.php** pada halaman **index.php,** anda cukup menggunakan method **include()** atau **require()** seperti berikut:

```
<!--
Nama    :
NIM     :
Kelas   :
-->

<?php
include ('header.php');
?>

    <main>
        Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
    </main>

<?php
include ('footer.php');
?>
```

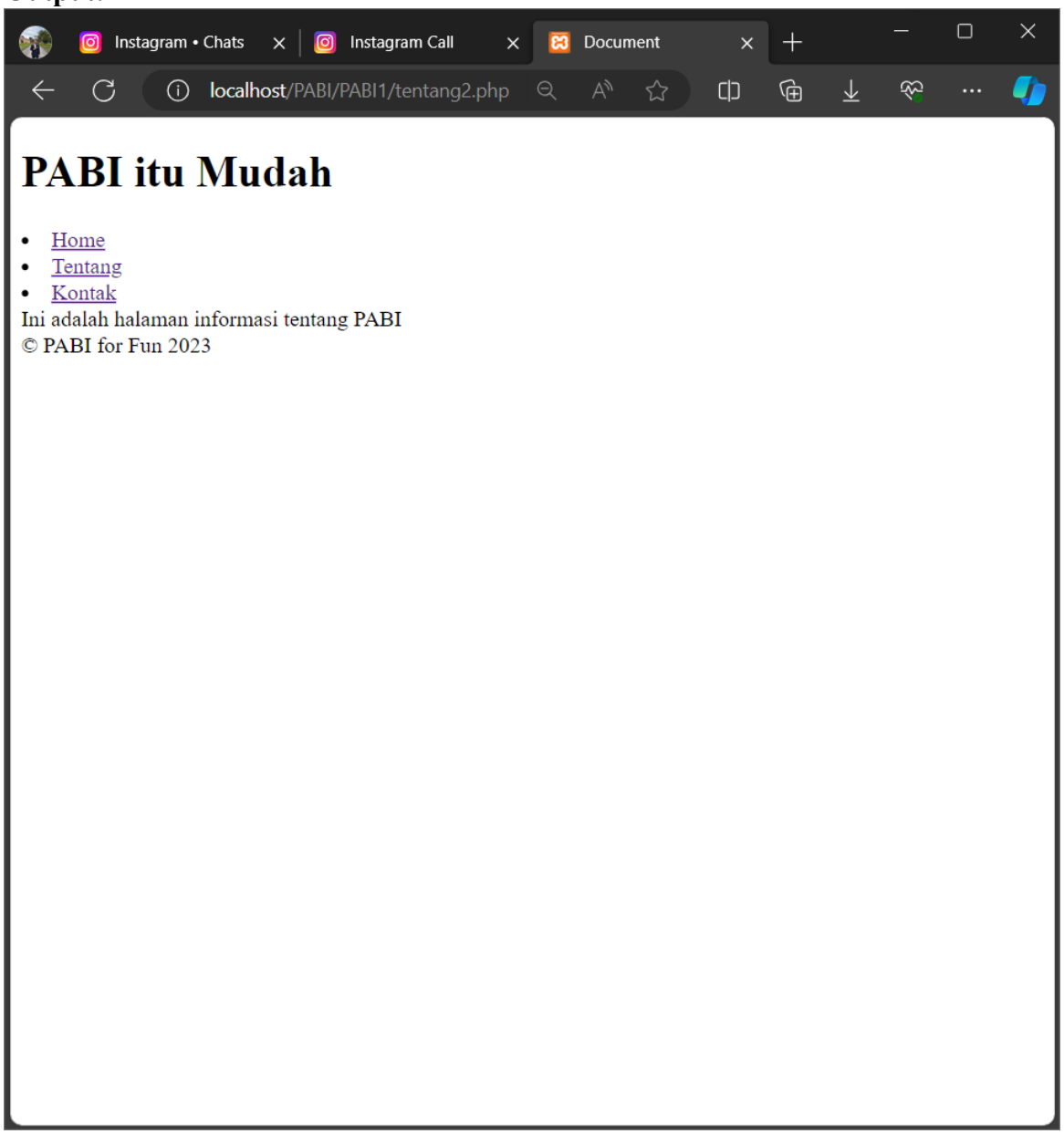
Klik **hyperlink** Home pada website, jika berhasil anda akan melihat konten header.php akan tertampil pada web-browser. Ditandai dengan tampilnya header: **"PABI itu Mudah".**

Output:

Selanjutnya silahkan ubah program anda sesuai dengan konten **index.php** dan tambahkan **include('header.php');** kedalam file php anda yang lain **(tentang.php dan kontak.php) dan** sehingga perubahannya menjadi seperti berikut pada **tentang.php**:

```
tentang.php > ...
<!--
Nama    :
NIM     :
Kelas   :
-->

<?php
include ('header.php');
?>
    <main>
        ini adalah halaman informasi tentang PABI
    </main>

<?php
include ('footer.php');
?>
```

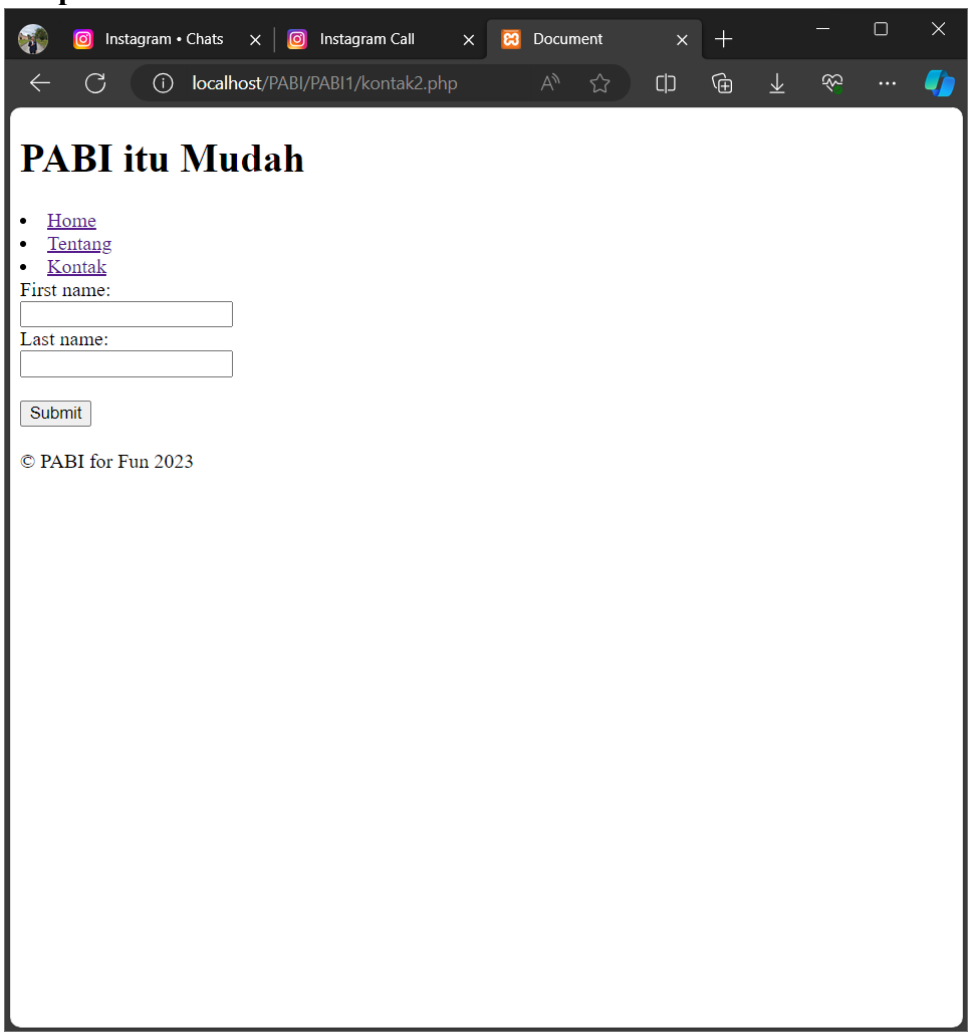
Output:



Ubahlah konten pada **kontak.php** menjadi seperti berikut:

```
kontak.php > ...
<!--
Nama    :
NIM     :
Kelas   :
-->

<?php
include ('header.php');
?>

    <main>
    <form action="">
        <label for="fname">First name:</label><br>
        <input type="text" id="fname" name="fname" value=""><br>
        <label for="lname">Last name:</label><br>
        <input type="text" id="lname" name="lname" value=""><br><br>
        <input type="submit" value="Submit"> <br><br>
    </form>
    </main>

<?php
include ('footer.php');
?>
```

Output:



sekarang user akan ditunjukkan pada praktikum ini, perbedaan antara **include()** dan **require().**

Perbedaannya adalah:

- **Include():** error akan ditampilkan dari baris program yang salah, tetapi baris program berikutnya masih tetap dieksekusi hingga baris program terakhir.
- **require():** error akan ditampilkan dari baris program yang salah, tetapi baris program berikutnya tidak akan dieksekusi dan program dipaksa berhenti. Biasanya digunakan untuk form yang bersifat mandatory (wajib) diisi, misalnya halaman **login** dan **register** sebelum user diarahkan ke halaman **utama/dashboard**.

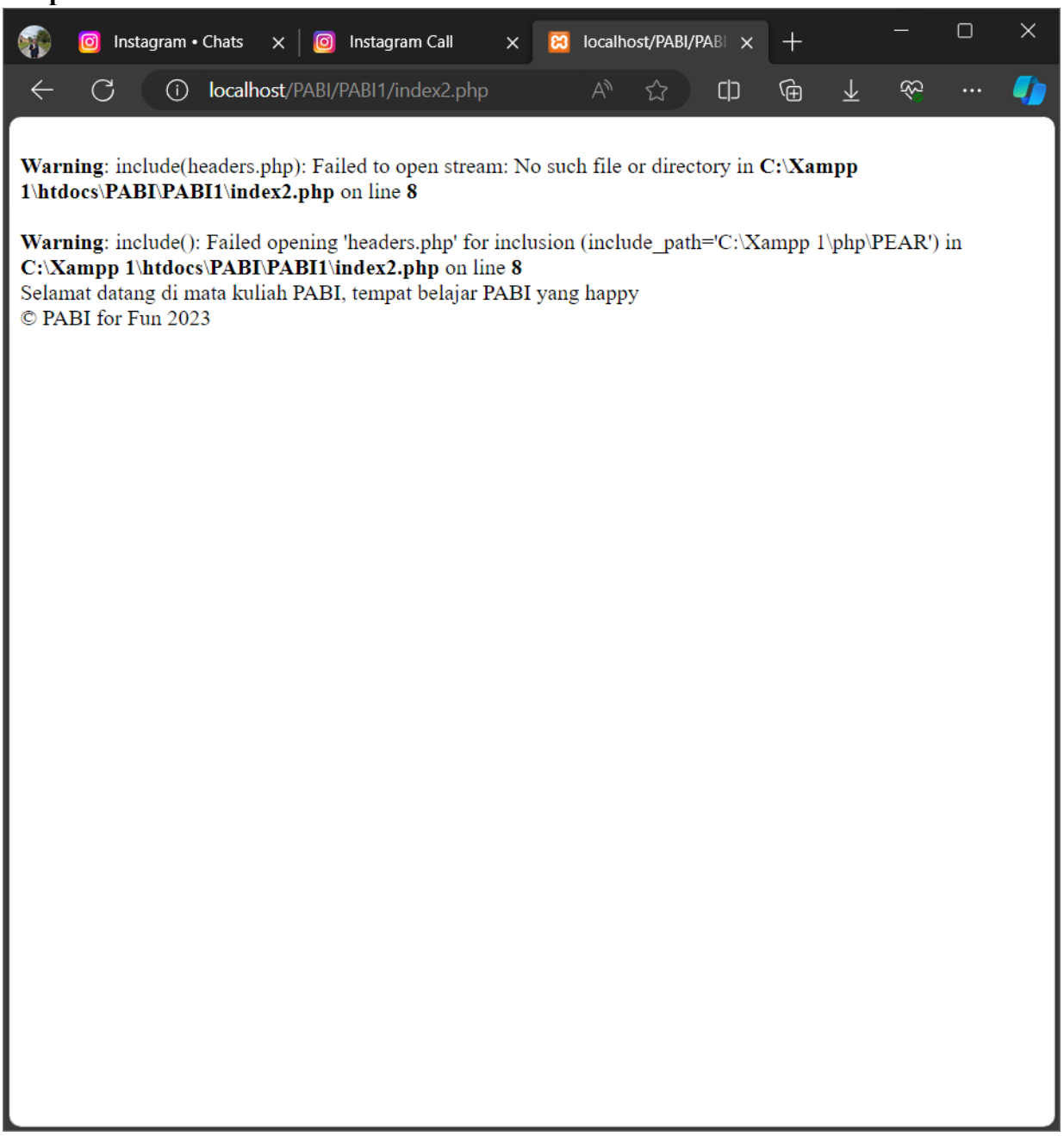
Dikarenakan bagian header dan footer bersifat wajib, maka anda disarankan menggunakan method require(). Silahkan ubah semua konten pada file **.php** anda dengan **require().**
Saya akan mencontohkan bagaimana perbedaan antara include() dan require() secara nyata.

Asumsikan anda memanggil konten pada file dengan nama yang salah. Misalnya, seharusnya nama file adalah **header.php** menjadi **headers.php** dan kombinasikan dengan include() dan require().

Contoh Include():

```
<!--
Nama    :
NIM     :
Kelas   :
-->

<?php
include ('headers.php');
?>

    <main>
        Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
    </main>

<?php
include ('footer.php');
?>
```

Output:



Web-browser akan menampikan eror dari baris program include ('headers.php');

Namun baris program selanjutnya masih tetap dieksekusi, yaitu:

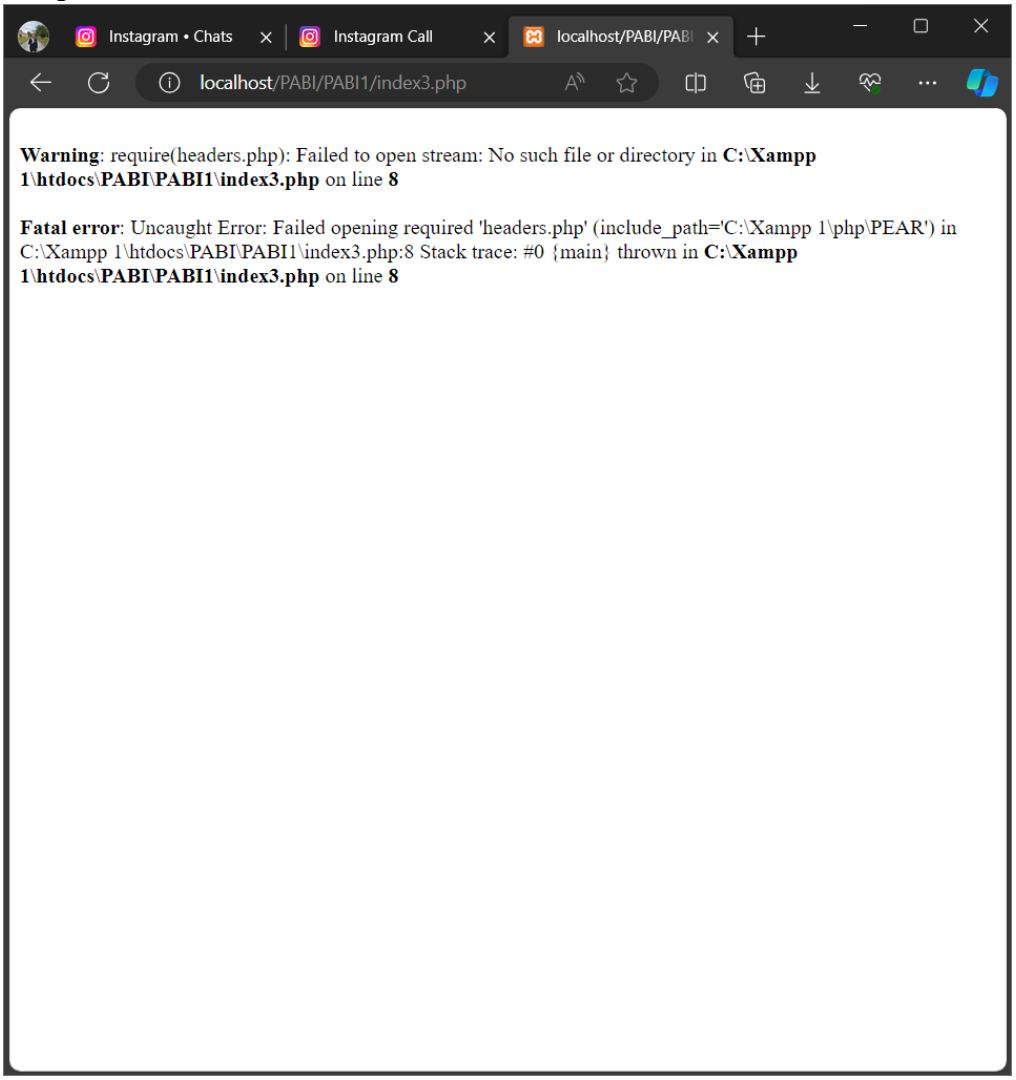
```
10
11      <main>
12          Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
13      </main>
14
15  <?php
16  include ('footer.php');
17  ?>
```

Mari kita bandingkan dengan menggunakan method **require(),** web-browser akan menampikan error dan tidak akan mengeksekusi baris program selanjutnya dan program dipaksa berhenti.

Contoh require():

```
index1.php > ...
1   <!--
2   Nama     :
3   NIM      :
4   Kelas    :
5   -->
6
7   <?php
8   require ('headers.php');
9   ?>
10
11      <main>
12          Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
13      </main>
14
15  <?php
16  include ('footer.php');
17  ?>
```

Output:



Jika menggunakan method **require()**, error ditampilkan oleh browser dan baris program berikutnya tidak akan dieksekusi dan program dipaksa berhenti jika terdapat eror.

Require_once() dan include_once()

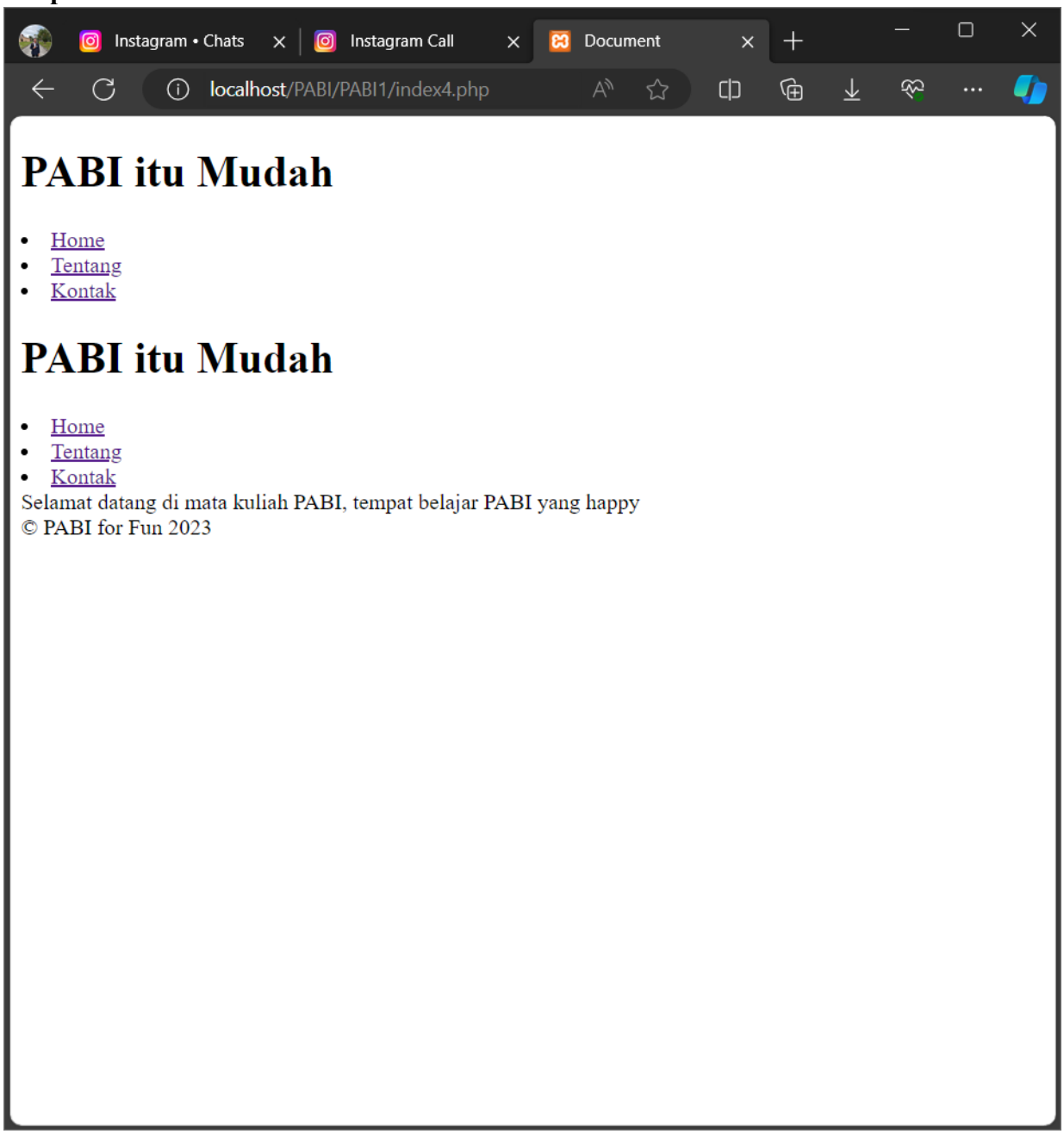
Require_once () dan include_once () digunakan untuk mencegah mengulang template sehingga template yang ditampikan persis tepat satu sekali.

Studi kasus: program index.php

```
7   <?php
8   require ('header.php');
9   ?>
10
11  <?php
12  require ('header.php');
13  ?>
14
15      <main>
16          Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
17      </main>
18
19  <?php
20  require ('footer.php');
21  ?>
```

Asumsikan anda tidak sengaja melakukan require() halaman header.php pada halaman index.php sebanyak 2 kali. Maka akan terdapat bug pada program dengan halaman yang berulang seperti output berikut.

Output:

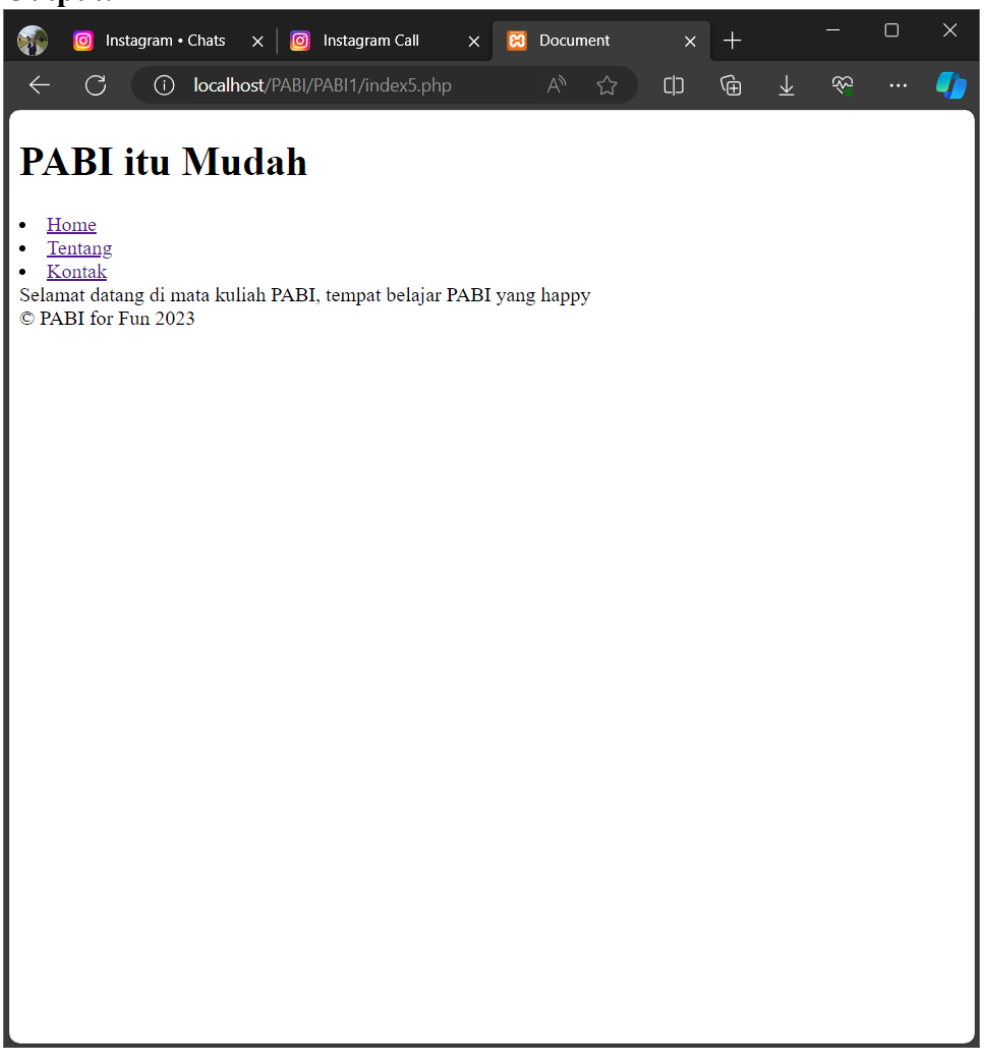


Solusinya anda cukup menggunakan method **require_once():**

```
week3 > index1.php > ...
1  <!--
2  Nama    :
3  NIM     :
4  Kelas   :
5  -->
6
7  <?php
8  require_once ('header.php');
9  ?>
10
11 <?php
12 require_once ('header.php');
13 ?>
14
15     <main>
16         Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
17     </main>
18
19 <?php
20 require_once ('footer.php');
21 ?>
```

Lihat apa yang terjadi?

Output:



Template halaman **header.php** dan **footer.php** hanya ditampilkan tepat satu kali walaupun **require_once()** ditulis berulang. Silahkan ubah semua file **PHP** anda dengan menggunakan method **require_once().**

Explode dan Implode

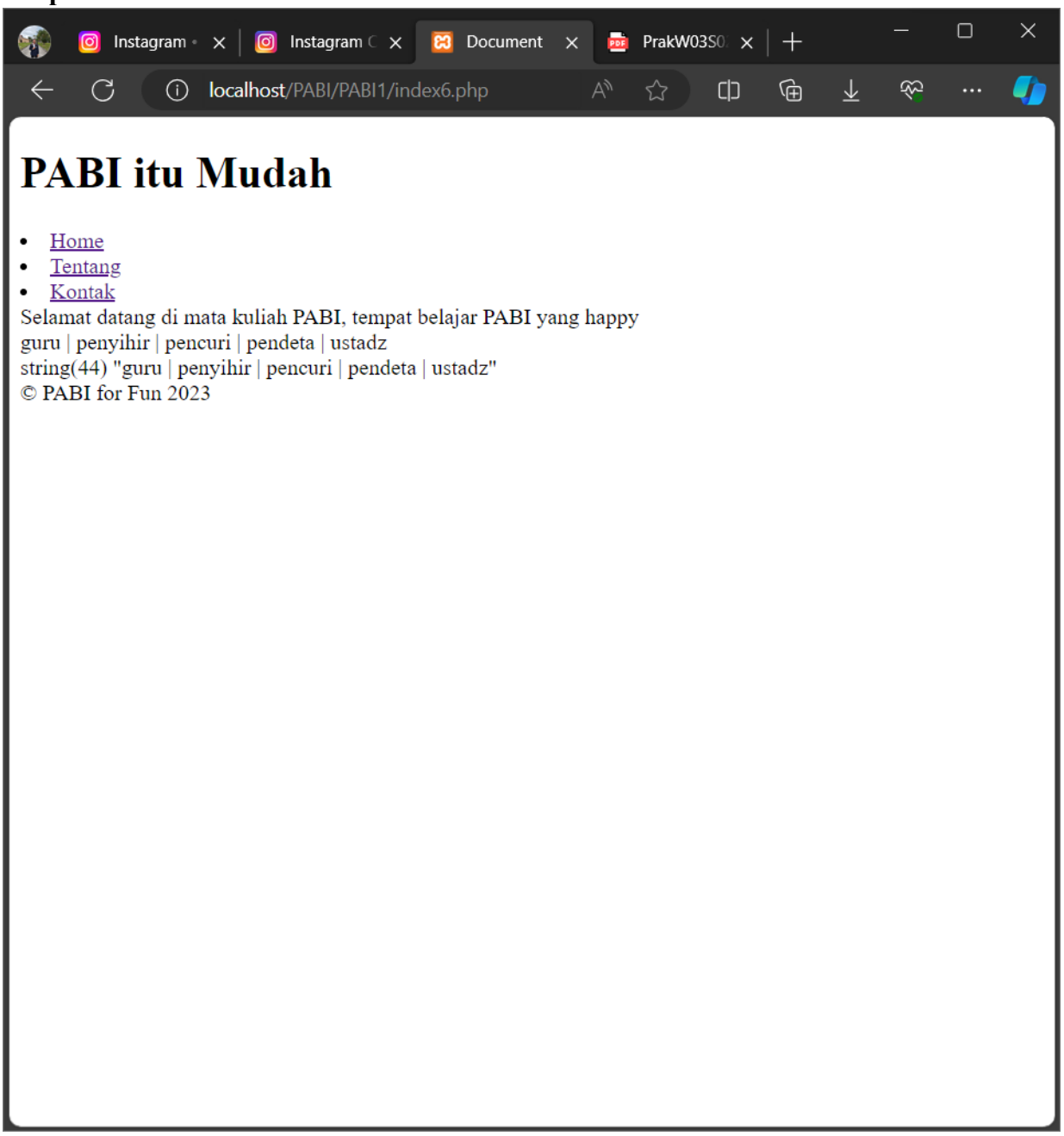
Perbedaan Explode dan Implode adalah:

- Explode: Mengubah format data dari **string** menjadi bentuk **array**.
- Implode: Kebalikannya, Mengubah format data dari **array** kedalam bentuk **string**.

Contoh Program **Implode** pada **index.php**:

```
eek3 > index1.php > ...
1  <!--
2  Nama    :
3  NIM     :
4  Kelas   :
5  -->
6
7  <?php
8  require_once ('header.php');
9  ?>
10     <main>
11         Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
12     </main>
13
14     <?php
15     $pekerjaan = ["guru", "penyihir", "pencuri", "pendeta", "ustadz"];
16     echo implode(" | ", $pekerjaan) . '<br>';
17     var_dump(implode(" | ", $pekerjaan));
18     echo '<br>';
19     ?>
20
21 <?php
22 require_once ('footer.php');
23 ?>
```

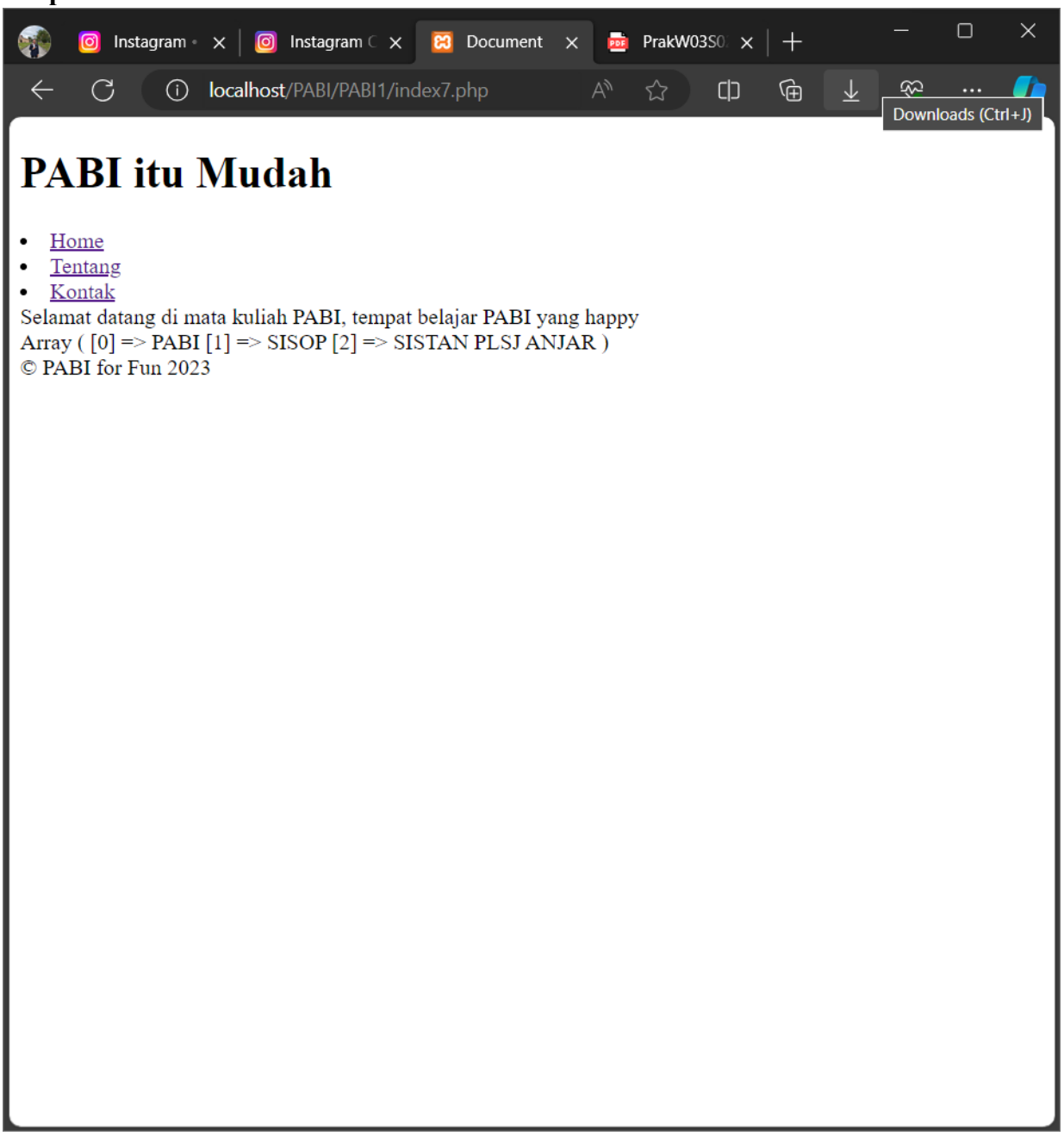
Output:


Instagram
Instagram
Document
PrakW03S0
localhost/PABI/PABI1/index6.php
PABI itu Mudah
Home
Tentang
Kontak
Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
guru | penyihir | pencuri | pendeta | ustadz
string(44) "guru | penyihir | pencuri | pendeta | ustadz"
© PABI for Fun 2023

Contoh Program **Explode** pada **index.php**:

```
week3 > index1.php > ...
1   <!--
2   Nama    :
3   NIM     :
4   Kelas   :
5   -->
6
7   <?php
8   require_once ('header.php');
9   ?>
10      <main>
11          Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
12      </main>
13
14      <?php
15      $pelajaran = "PABI SISOP SISTAN PLSJ ANJAR";
16      print_r(explode(" ",$pelajaran, 3));
17      ?>
18
19  <?php
20  require_once ('footer.php');
21  ?>
```

Output:


Instagram
Instagram
Document
PrakW03S0
localhost/PABI/PABI1/index7.php
Downloads (Ctrl+J)
PABI itu Mudah
Home
Tentang
Kontak
Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
Array ([0] => PABI [1] => SISOP [2] => SISTAN PLSJ ANJAR)
© PABI for Fun 2023

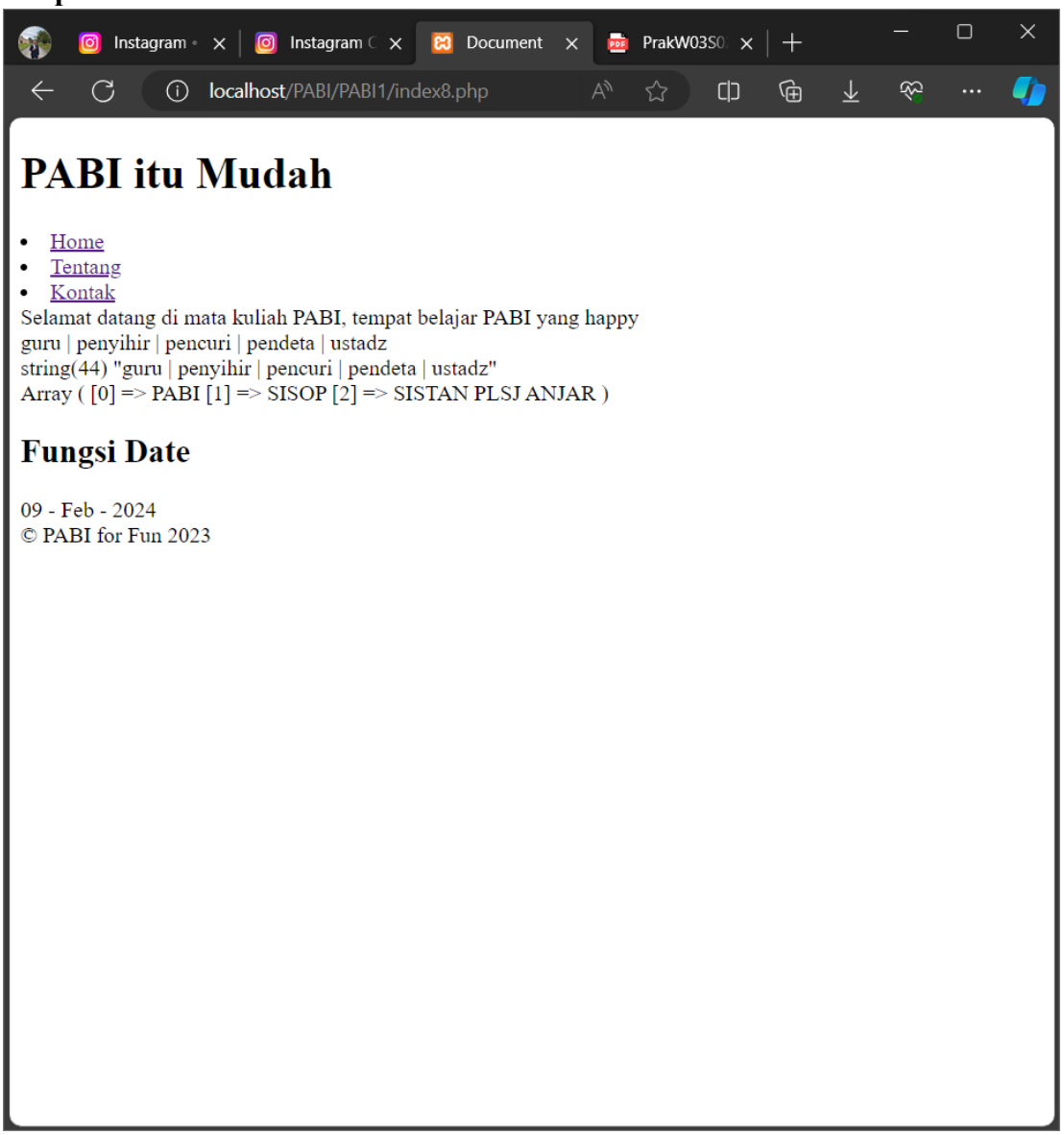
Fungsi date

Referensi: https://www.php.net/manual/en/function.date.php

Contoh Program menggunakan fungsi date() pada index.php:

```php
<?php
require_once ('header.php');
?>
    <main>
        Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
    </main>

    <?php
    $pekerjaan = ["guru", "penyihir", "pencuri", "pendeta", "ustadz"];
    echo implode(" | ", $pekerjaan) . '<br>';
    var_dump(implode(" | ", $pekerjaan));
    echo '<br>';

    $pelajaran = "PABI SISOP SISTAN PLSJ ANJAR";
    print_r(explode(" ",$pelajaran, 3));
    ?>

    <h2>Fungsi Date</h2>

    <?php
    echo date('d - M - Y');
    ?>

<?php
require_once ('footer.php');
?>
```

Output:

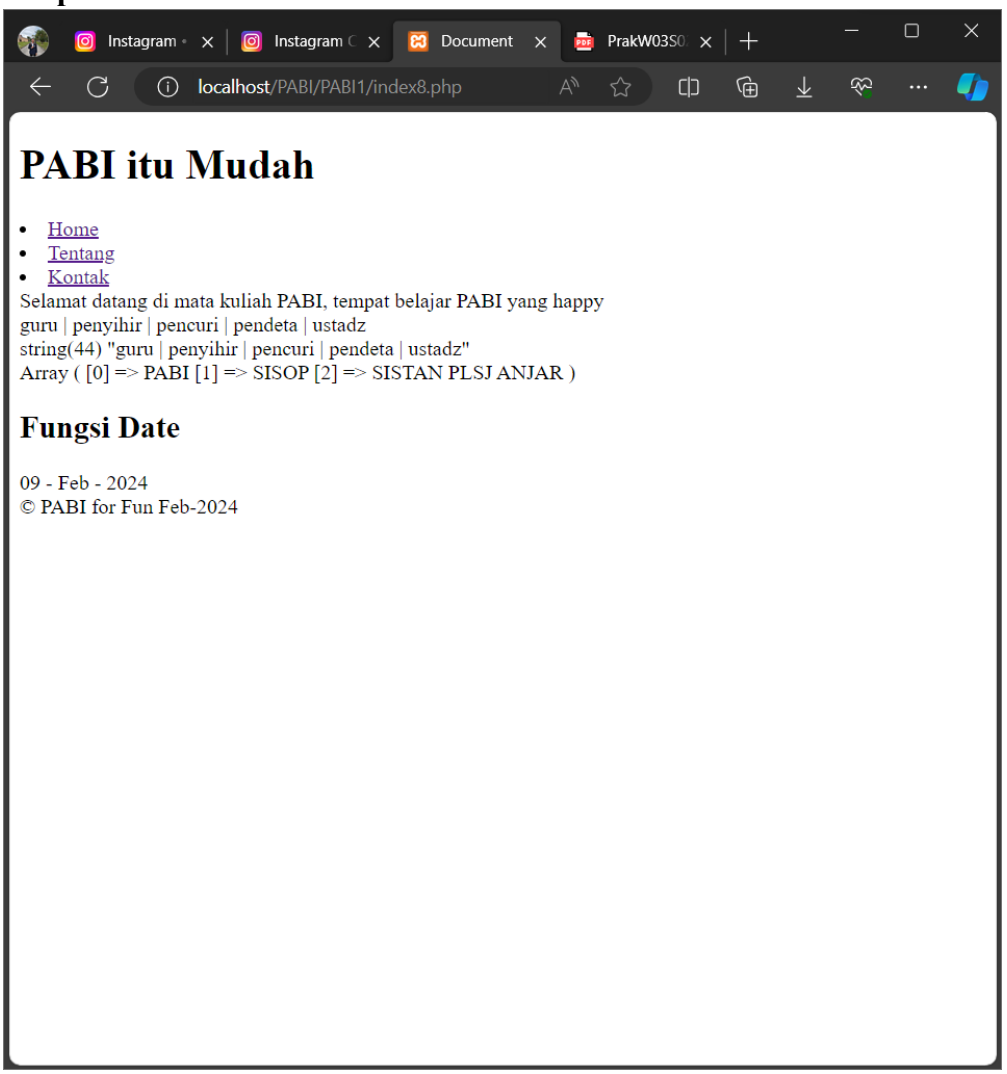


Sekarang anda dapat membuat konten **tahun** pada halaman **footer.php** anda menjadi dinamis menggunakan fungsi **date.**

Silahkan ubah **"2022"** yang sebelumnya diketik manual, menjadi otomatis menggunakan fungsi **date()** seperti berikut:

```
6
7       <footer>
8           &copy; PABI for Fun
9
10          <?php
11          echo date ('M-Y')
12          ?>
13
14      </footer>
15  </body>
16  </html>
```

Output:


Instagram
Instagram
Document
PrakW03S0
localhost/PABI/PABI1/index8.php
PABI itu Mudah
Home
Tentang
Kontak
Selamat datang di mata kuliah PABI, tempat belajar PABI yang happy
guru | penyihir | pencuri | pendeta | ustadz
string(44) "guru | penyihir | pencuri | pendeta | ustadz"
Array ([0] => PABI [1] => SISOP [2] => SISTAN PLSJ ANJAR)
Fungsi Date
09 - Feb - 2024
© PABI for Fun Feb-2024

Trim() dan strip_tags

Fungsi trim() yang digunakan untuk menghapus spasi (whitespace) di awal dan akhir sebuah string PHP. Fungsi trim() juga memiliki 2 varian lain yakni ltrim() dan rtrim().

Pengertian Fungsi trim()

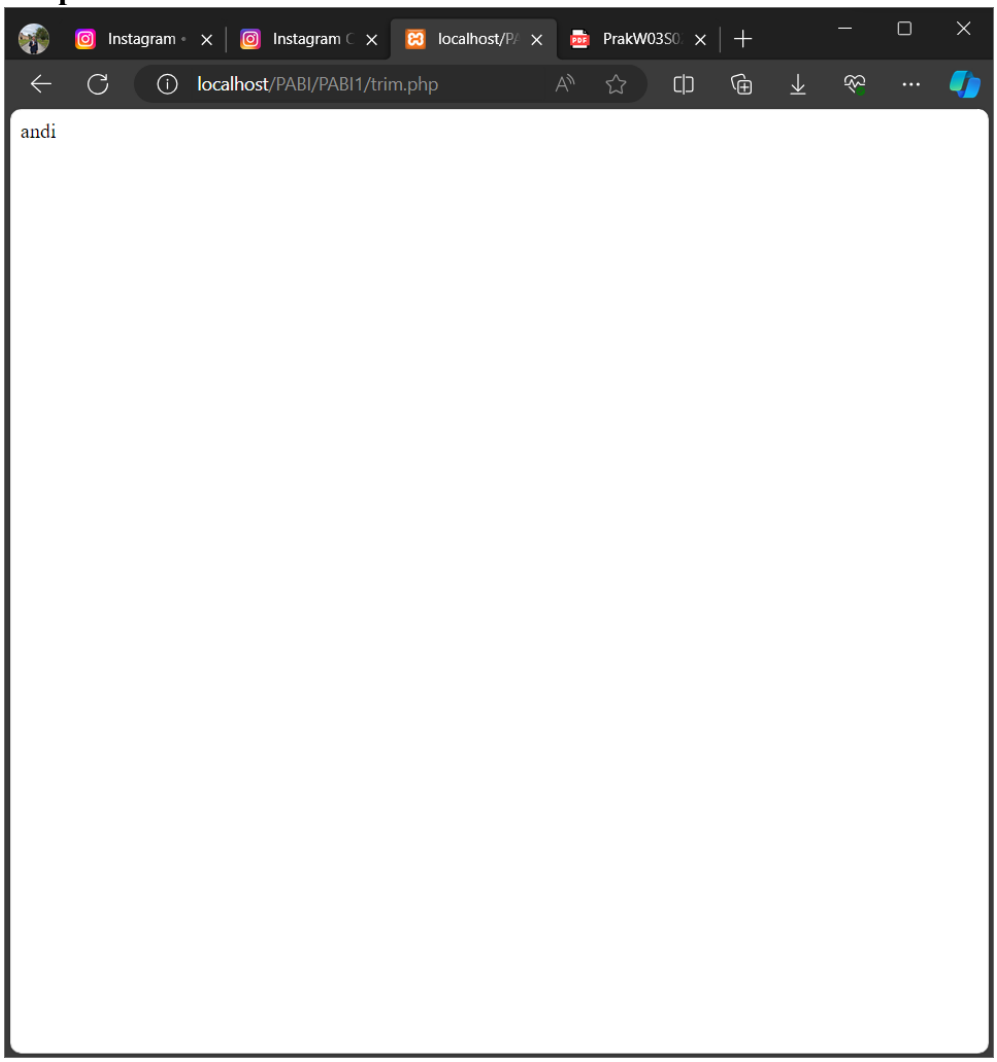
Secara default bawaan PHP, fungsi trim() digunakan untuk menghapus spasi atau karakter whitespace dari sebuah string. Karakter spasi yang akan dihapus bisa berada di awal maupun di akhir string.

Dalam prakteknya, fungsi trim() sering digunakan untuk 'membersihkan' hasil input form dari karakter spasi yang sengaja atau tidak sengaja ditambahkan pengguna.

Berikut adalah contoh penggunaan dasar fungsi trim() di dalam PHP:

```php
<?php
    $nama = " andi      ";
    $trim_nama = trim($nama);
    echo $trim_nama; // "andi"
?>
```

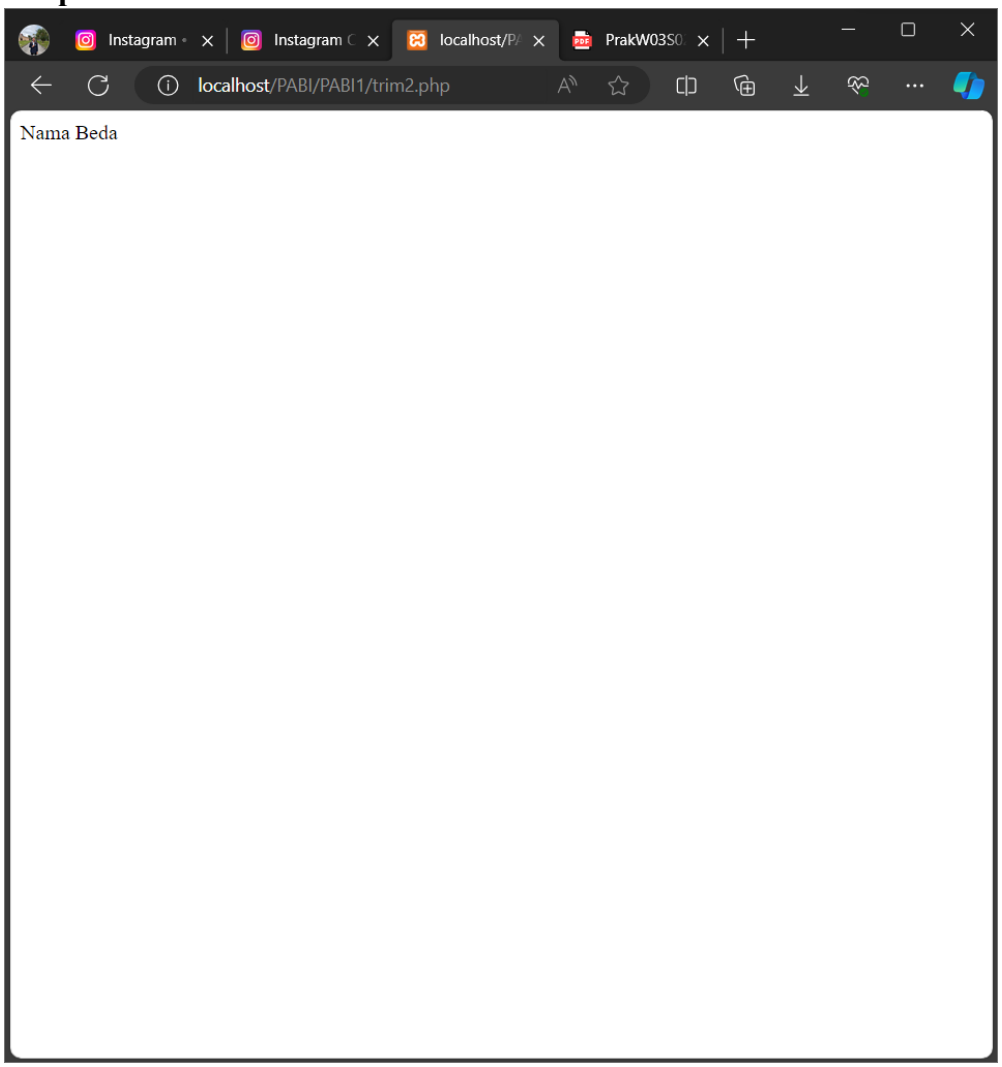
Output:



Karena di HTML whitespace atau spasi tidak akan ditampilkan, penerapan kode diatas tidak terlalu jelas efeknya. Fungsi trim() akan lebih terlihat jika digunakan dalam operasi perbandingan, seperti contoh berikut:

```php
<?php
    $nama = "andi ";
    $nama_juga = " andi";
    if ($nama == $nama_juga) {
        echo "Nama Sama";
    }
    else {
        echo "Nama Beda";
    }
    // hasil: Nama Beda
?>
```

Output:

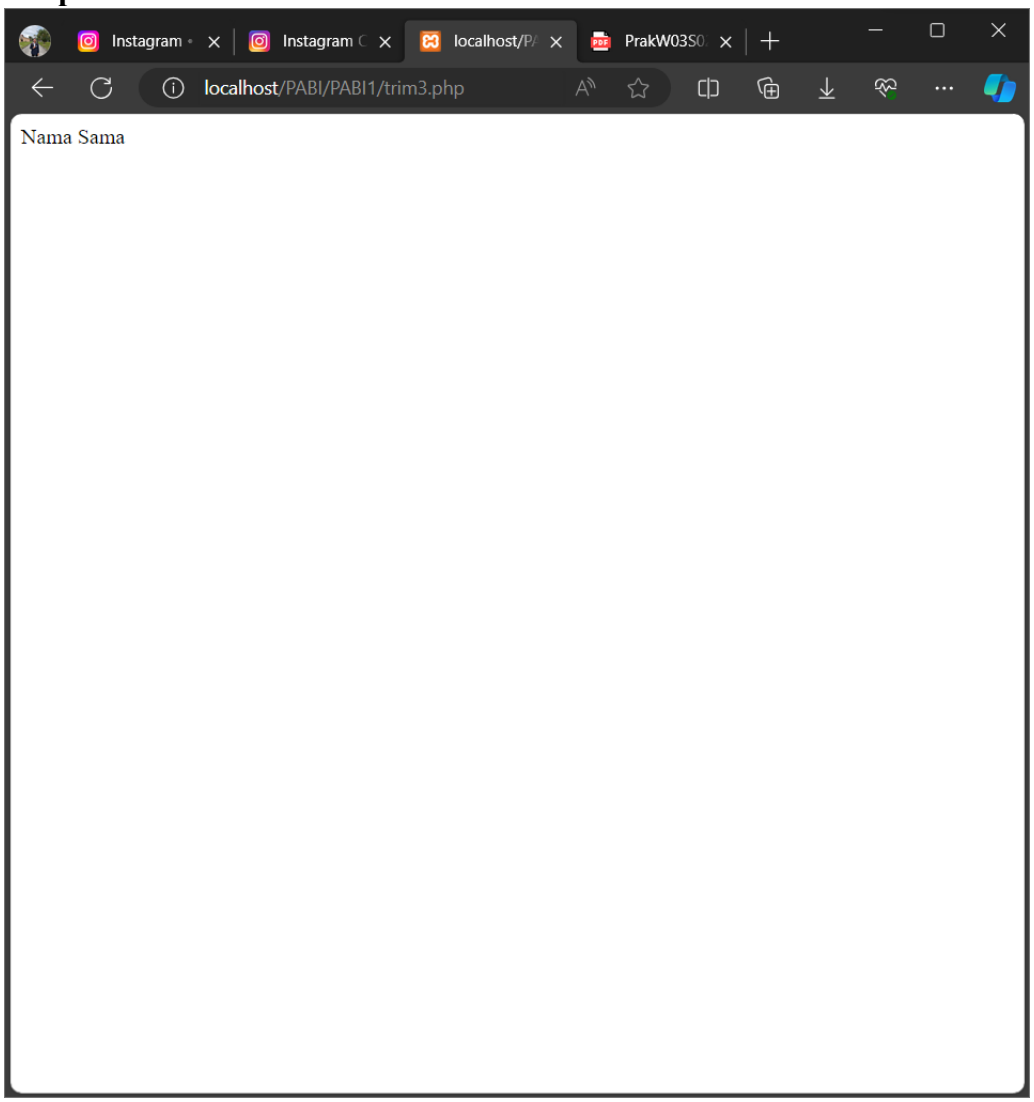


Dalam operasi perbandingan diatas, tambahan sebuah spasi diakhir variabel $nama, yakni “andi ” akan membuat operasi perbandingan menghasilkan nilai FALSE, sehingga hasil akhirnya adalah “Tidak Sama”.

Dengan menambahkan fungsi trim(), kode program diatas akan menghasilkan nilai TRUE, karena spasi yang ada baik diawal dan diakhir string akan dihapus terlebih dahulu:

```php
<?php
    $nama = "andi ";
    $nama_juga = " andi";
    if (trim($nama) == trim($nama_juga)) {
        echo "Nama Sama";
    }
    else {
        echo "Nama Beda";
    }
    // hasil: Nama Sama
?>
```

Output:



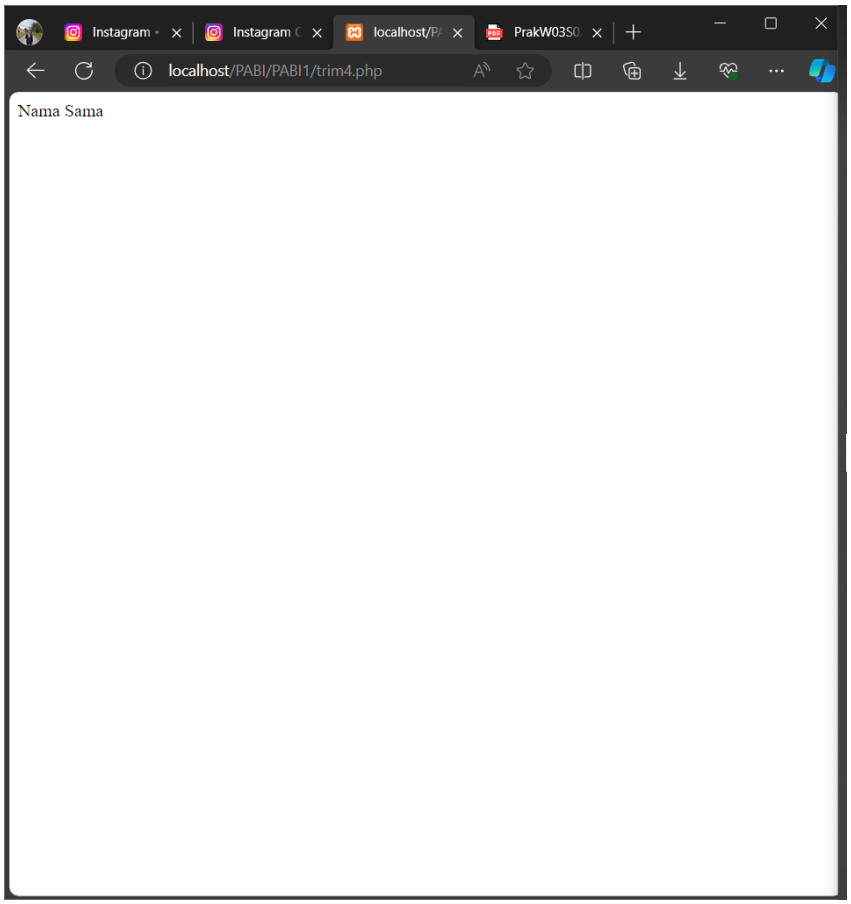
Selain menghapus karakter spasi, fungsi trim() juga akan menghapus 5 karakter whitespace lainnya, seperti tab, new line, carriage return (karakter enter), null-byte, dan vertical tab. Dalam kode karakter ASCII, ke-6 karakter ini adalah sebagai berikut:

- ” ” (ASCII 32 (0x20)), : karakter spasi.
- “\t” (ASCII 9 (0x09)), : karakter tab.
- “\n” (ASCII 10 (0x0A)), : karakter new line (line feed).
- “\r” (ASCII 13 (0x0D)), : karakter carriage return.
- “\0” (ASCII 0 (0x00)), : karakter NULL-byte.
- “\x0B” (ASCII 11 (0x0B)), : karakter vertical tab.

Berikut contoh penggunaannya:

```php
<?php
    $nama = "\t \t andi \n \r";
    $nama_juga = "    andi \t";
    if (trim($nama) == trim($nama_juga)) {
        echo "Nama Sama";
    }
    else {
        echo "Nama Beda";
    }
    // hasil: Nama Sama
?>
```

Output:

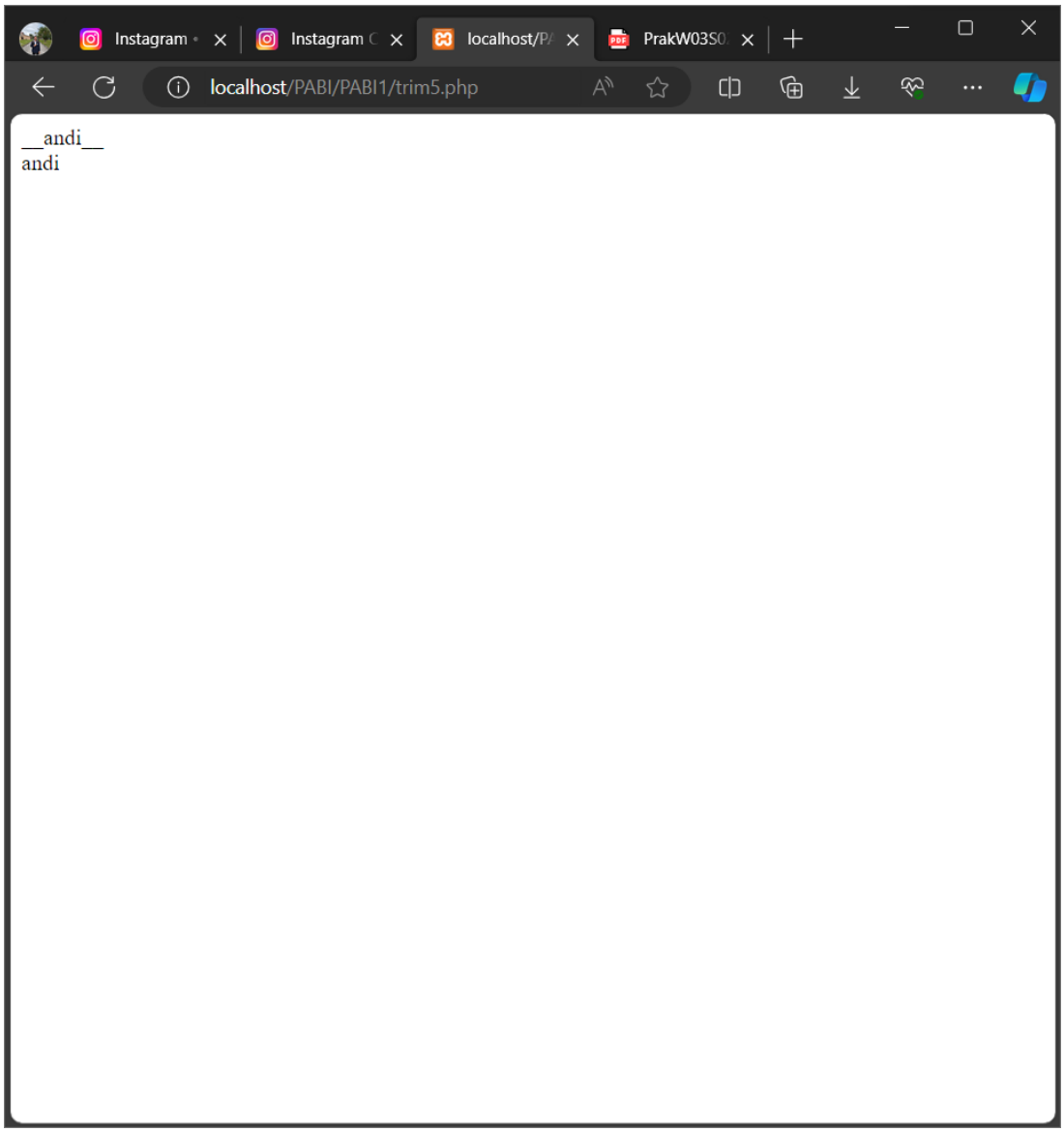


Untuk menulis karakter “tab”, tidak tersedia tombol khusus di dalam keyboard, oleh karena itu kita menggunakan escape karakter untuk tab, yakni “\t”.

Fungsi trim() juga memiliki argumen kedua yang bersifat opsional. Argumen kedua ini bertipe string yang jika ditulis akan ditambahkan kedalam daftar karakter yang ikut dihapus, atau istilah teknisnya disebuh dengan **character_mask**. Langsung saja kita lihat contoh penggunaannya:

```php
<?php
    $nama = "__andi__";
    $trim_nama = trim($nama);
    echo $trim_nama; // "__andi__"
    echo "<br>";

    $trim_nama = trim($nama,"_");
    echo $trim_nama; // "andi"
?>
```

Output:

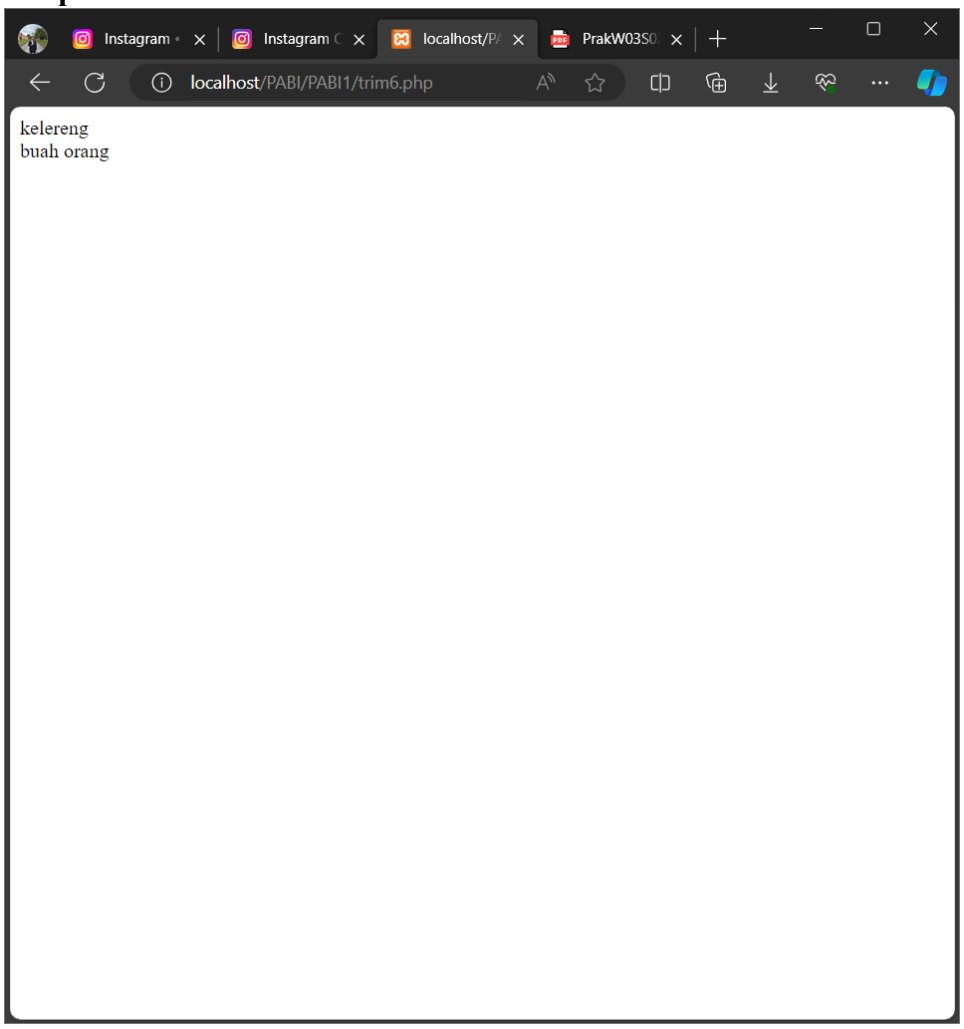


Lebih jauh lagi, character_mask ini mendukung penulisan range atau jangkauan karakter, yang ditulis dengan “awal..akhir”.

Sebagai contoh, jika saya memiliki beberapa string yang dimulai dengan angka seperti: “1 kelereng”, “2 buah”, “3 orang”, saya bisa menggunakan fungsi **trim()** untuk menghapus seluruh angka awalan ini. Berikut contohnya:

```
6
7   <?php
8       $kata = "1 kelereng";
9       $trim_kata = trim($kata,"0..9");
10      echo $trim_kata; // "kelereng"
11      echo "<br>";
12
13      $kata = "2 buah";
14      $trim_kata = trim($kata,"0..9");
15      echo $trim_kata; // "buah"
16      echo "<br>";
17
18      $kata = "3 orang";
19      $trim_kata = trim($kata,"0..9");
20      echo $trim_kata; // "orang"
21  ?>
```

Output:



Fungsi **trim($kata,"0..9″)** berarti: hapus whitespace yang ada di awal dan akhir string $kata, dan hapus karakter 0, 1, 2, 3 s/d 9 di awal dan diakhir string. Fitur untuk menghapus karakter tertentu ini cukup berguna ketika kita ingin 'membersihakan' sebuah string dari karakter yang tidak diinginkan.

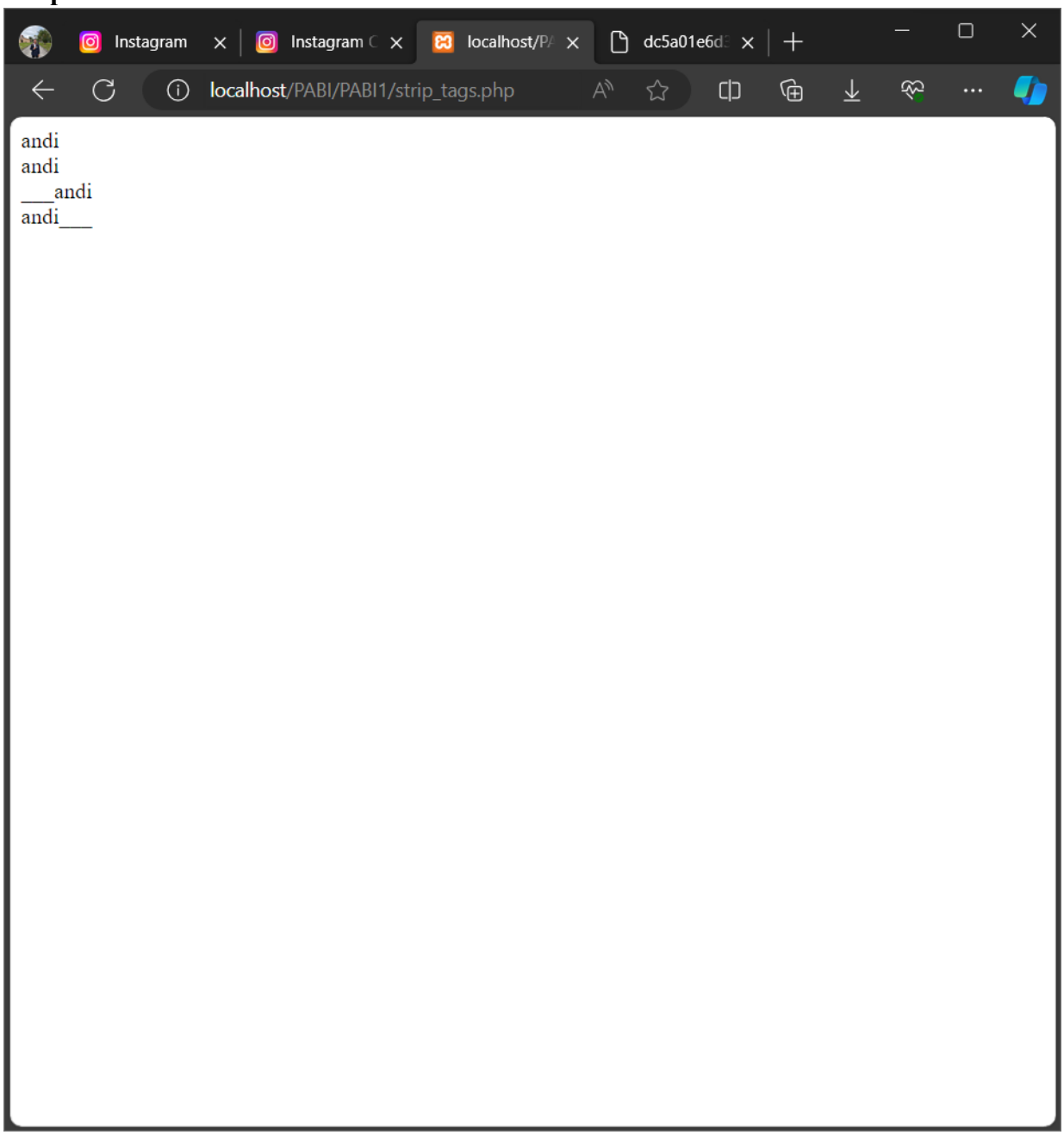
Pengertian Fungsi rtrim() dan ltrim()

Fungsi rtrim() dan ltrim() adalah bentuk lain dari fungsi trim(), tapi hanya akan menghapus karakter whitespace yang ada disisi kanan (untuk rtrim) dan di sisi kiri string (untuk ltrim). Kedua fungsi ini juga bisa ditambahkan argumen ketiga seperti halnya fungsi trim().

Berikut contoh penggunaan fungsi rtrim() dan ltrim() dalam PHP:

```
7   <?php
8       $nama = " andi ";
9
10      $rtrim_nama = rtrim($nama);
11      echo $rtrim_nama; // " andi"
12      echo "<br>";
13
14      $ltrim_nama = ltrim($nama);
15      echo $ltrim_nama; // "andi "
16      echo "<br>";
17
18      $nama1 = "___andi___";
19
20      $rtrim_nama1 = rtrim($nama1,"_");
21      echo $rtrim_nama1; // "___andi"
22      echo "<br>";
23
24      $ltrim_nama1 = ltrim($nama1,"_");
25      echo $ltrim_nama1; // "andi___"
26  ?>
27
```

Output:



Dapat terlihat bahwa fungsi **rtrim()** dan fungsi **ltrim()** hanya akan menghapus karakter pada satu sisi saja.

Fungsi trim(), rtrim() dan ltrim() yang kita pelajari disini cocok digunakan untuk operasi 'pembersihan' string, ini umumnya digunakan untuk menfilter hasil inputan form dari pengguna (user).

Fungsi strip_tags() dengan htmlentities()

Jika anda melihat sebuah website pada bagian kolom komentar, sering para pengunjung meletakkan nama urlnya dengan menggunakan tag HTML **<a href="namadomain">Kalimat</a>,** dengan maksud agar visitor lain dapat mengklik linknya dan menuju suatu situs tertentu. Nah sekarang bagaimana agar setiap pengunjung yang mengisi komentar hanya dapat mengisi teks biasa, jika memasukkan tag HTML maka akan dihapus bagian tag HTMLnya aja.

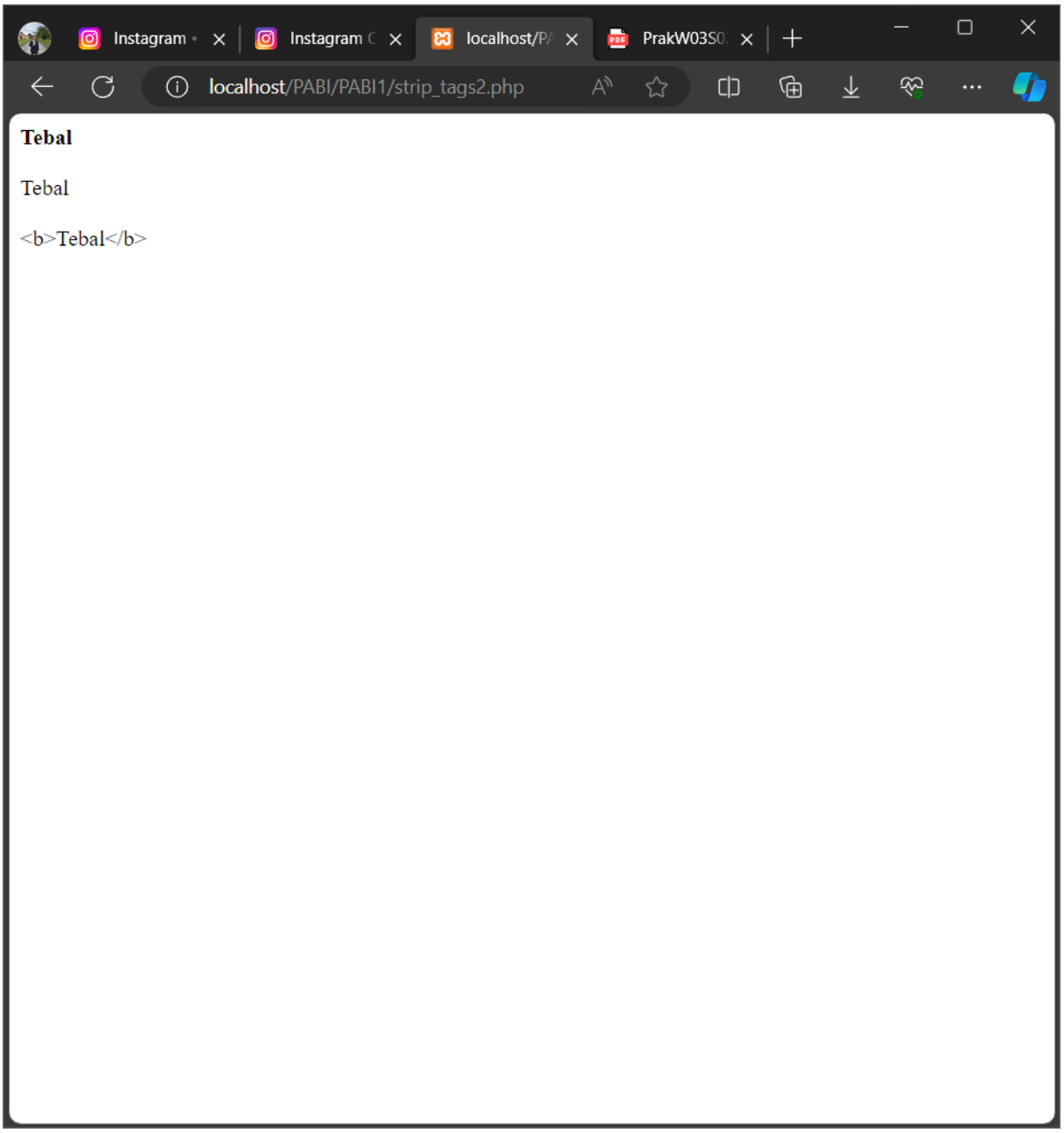
Supaya isi dari kolom komentar dianggap teks biasa, ada dua teknik yang dapat kita lakukan. Yang pertama dengan menggunakan fungsi **strip_tags(),** yang kedua dengan menggunakan fungsi **htmlentities().**

Perbedaan antara fungsi strip_tags() dengan htmlentities() ditunjukkan oleh contoh dibawah ini:

week3 > strip_tags.php > ...

```php
<?php
    $kalimat = "<b>Tebal</b>";
    echo $kalimat."<br><br>";
    pakai_strip_tags();
    pakai_htmlentites();

    function pakai_strip_tags()
    {
        GLOBAL $kalimat;
        $filter1 = strip_tags($kalimat);
        echo $filter1."<br><br>";
    }
    function pakai_htmlentites()
    {
        GLOBAL $kalimat;
        $filter2 = htmlentities($kalimat);
        echo $filter2;
    }
?>
```

Outputnya:



Seperti yang kita lihat, apabila terdapat sebuah kalimat yang menggunakan tag HTML , maka kalimat tersebut akan ditampilkan dengan huruf tebal, yaitu : Tebal

Nah karena kita gunakan fungsi strip_tags(), maka tag HTML tersebut akan dihapus, sehingga hanya tertampilkan kalimat tanpa mengalamai penebalan, yaitu : Tebal.

Tetapi jika kita menggunakan fungsi htmlentities(), maka tag HTML akan dianggap teks biasa, sehingga akan ditampilkan : Tebal.

Dalam pembahasan berikutnya kita akan fokus tata cara penggunaan fungsi strip_tags() berserta contoh penerapannya.

Pengenalan
Seperti yang dijelaskan diatas, fungsi **strip_tags()** bertujuan untuk menghilangkan tag HTML. Fungsi ini mulai diperknalkan pada **PHP versi 4+** dan masih digunakan hingga sekarang. Syntax dari penulisan fungsi substr() :

strip_tags(string, allowable_tags)

Keterangan:

- string mengacu kepada sebuah kalimat yang akan diperiksa (Required)
- allowable_tags mengacu kepada tag HTML yang dibolehkan (Opstional)

Menghilangkan semua tag HTML
Misalkan kita memiliki sebuah kalimat : Pusat Ilmu Secara Detil
Jika kita melihat kalimat tersebut, maka yang dibold adalah Pusat dan Detil, sedangkan yang italic adalah Ilmu Secara.

Untuk membuat bold dalam HMTL kita menggunakan tag Untuk membuat italic dalam HTML kita menggunakan tag

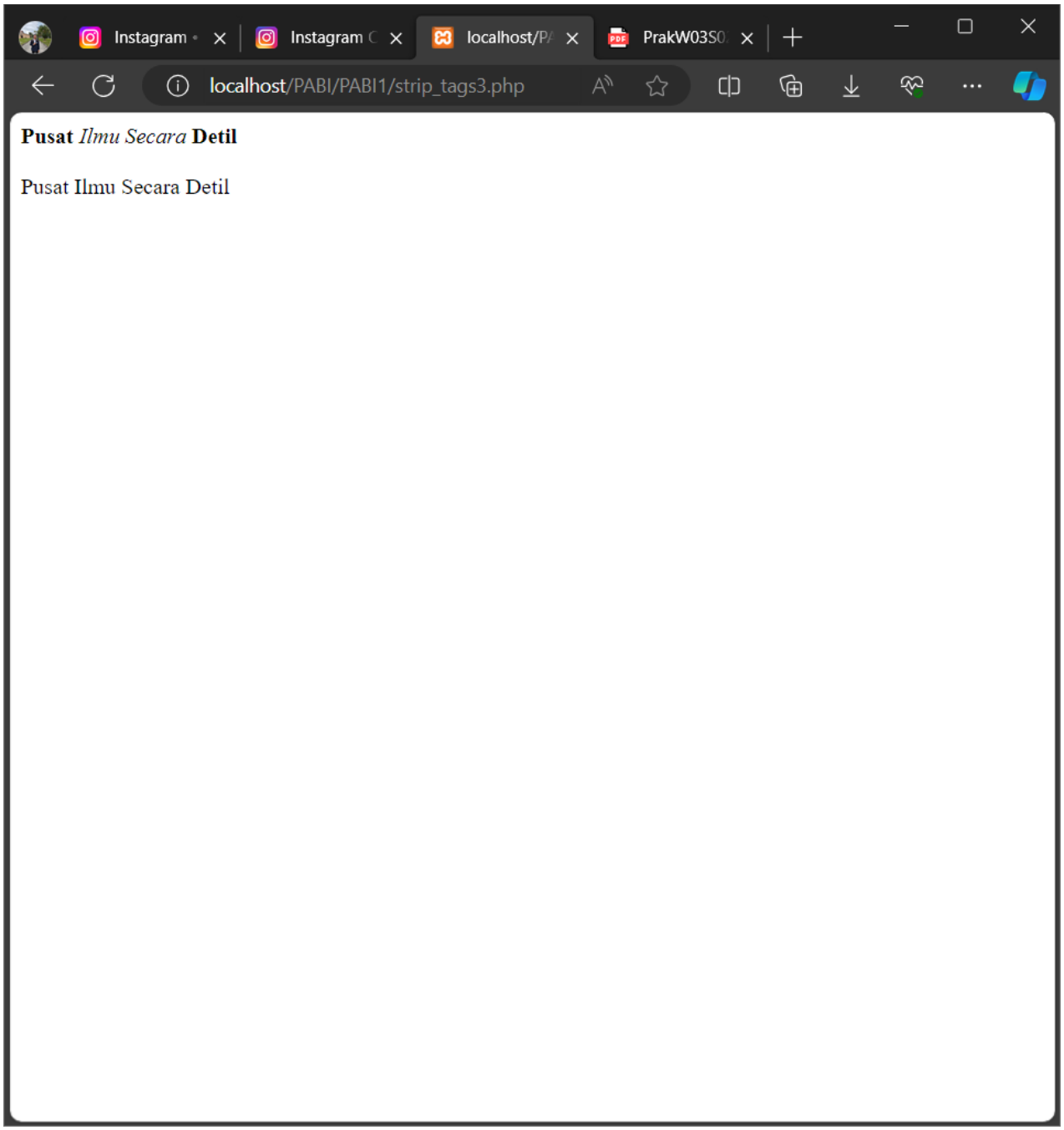
Nah sekarang kita akan tunjukkan bagaimana menghilangkan kedua tag HTML tersebut, yaitu: tag serta tag , sehingga kalimat yang ditampilkan adalah:
Pusat Ilmu Secara Detiltanpa ada penebalan (bold) pada Pusat dan Detil serta tanpa ada pemiringan pada kalimat Ilmu Secara.

Perhatikan Contoh penggunaannya untuk memahami lebih lanjut:

week3 > strip_tags.php > ...

```
1   <?php
2       tanpa_strip_tags();
3       memakai_strip_tags();
4       function tanpa_strip_tags()
5       {
6           $kalimat = "<b>Pusat</b> <i>Ilmu Secara</i> <b>Detil</b>";
7           echo $kalimat;
8           echo"<br><br>";
9       }
10      function memakai_strip_tags()
11      {
12          $kalimat = "<b>Pusat</b> <i>Ilmu Secara</i> <b>Detil</b>";
13          echo strip_tags($kalimat);
14      }
15  ?>
```

Outputnya akan ditunjukkan oleh gambar dibawah ini:



Menghilangkan tag HTML tertentu

Kita masih menggunakan kalimat diatas yaitu: Pusat Ilmu Secara Detil.
Sekarang kita akan menghilangkan bagian yang ditebalkan pada kata Pusat dan Detil. Untuk itu kita akan mengizinkan tag dan membuang tag.

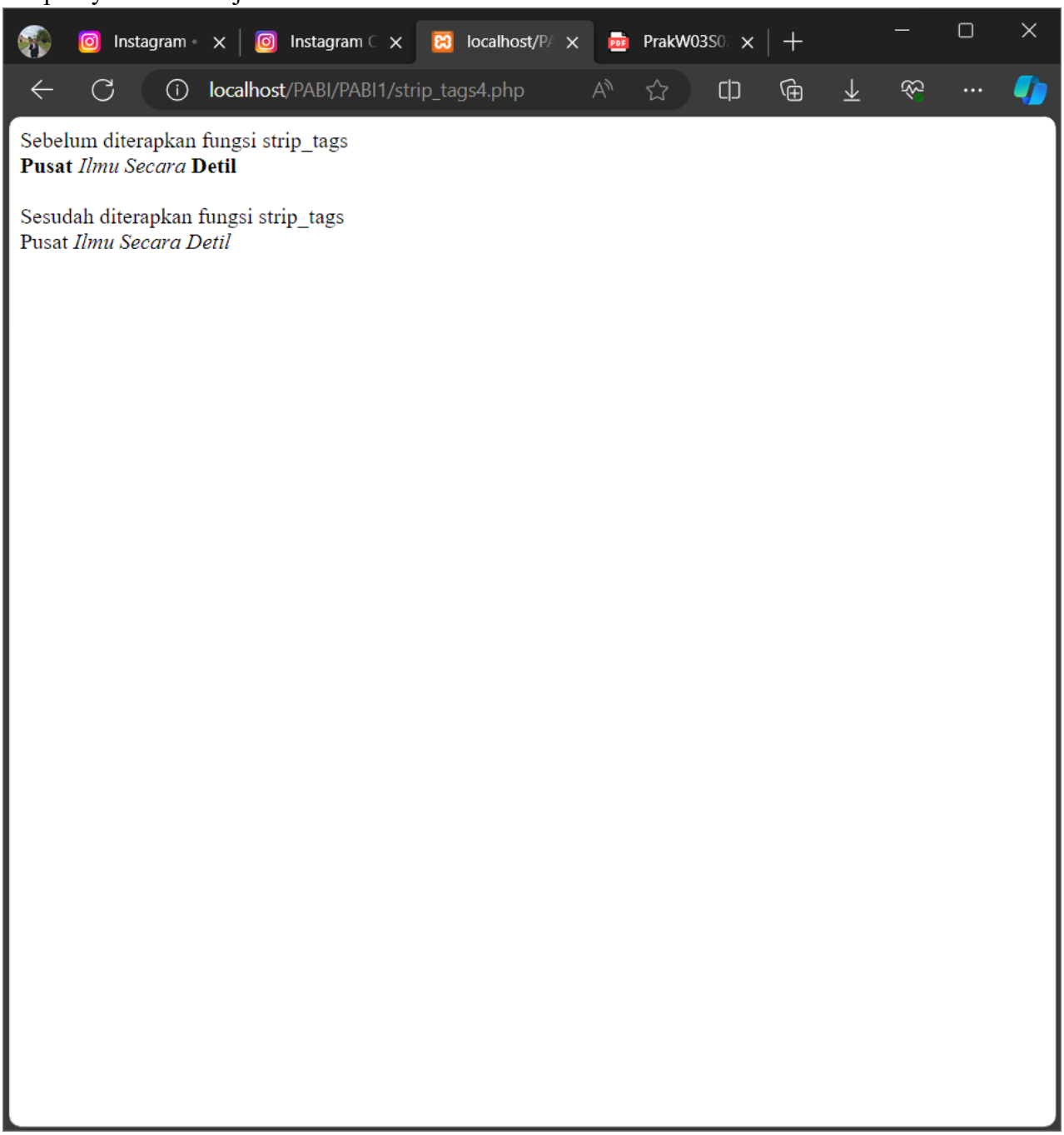
Sehingga output yang kita harapkan adalah:
Pusat Ilmu Secara Detil

Perhatikan contoh berikut untuk memahami bagaimana menghilangkan tag HTML tertentu:

week3 > strip_tags.php > ...

```php
<?php
    tanpa_strip_tags();
    memakai_strip_tags();
    function tanpa_strip_tags()
    {
        echo "Sebelum diterapkan fungsi strip_tags"."<br>";
        $kalimat = "<b>Pusat</b> <i>Ilmu Secara</i> <b>Detil</b>";
        echo $kalimat;
        echo"<br><br>";
    }
    function memakai_strip_tags()
    {
        echo "Sesudah diterapkan fungsi strip_tags"."<br>";
        $kalimat = "<b>Pusat</b> <i>Ilmu Secara</i> <b>Detil</b>";
        echo strip_tags($kalimat,"<i>");
    }
?>
```

Outputnya akan ditunjukkan oleh Gambar dibawah ini:



"At forty, I was too old to work as a programmer myself anymore; writing code is a young person's job."

Michael Crichton